

От размещения до подтверждённой продажи

Задача. Планировать маркетинг, сохранять источник каждого поста и сравнивать экономику подтверждённых оплат. Команда «Наступашки». Три экрана: **Календарь** / **Воронка** / **Результаты**. CRM и ручные стадии убраны.

Материалы кейса и границы данных

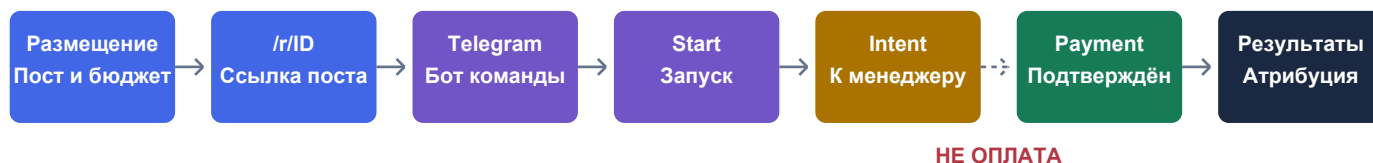
795 строк оплат · 606 анонимных ID · 18 курсов.

Telegram: 182 наблюдаемых поста пяти каналов, 62 проверенных в календаре. Colab: сохранены 23 исходные EDA-ячейки.

XLSX: курс, сумма и время за 04.08-10.09.2026. Часть сумм приближена, часовой пояс неизвестен, строка курса не обязательно заказ. Нет исторических лидов, рекламных расходов и источников покупателей. Telegram - наблюдаемый снимок, полнота архива не доказана.

Синтетическое демо: 13 размещений, 94 web-события, 150 bot-событий (96 start / 54 payment_click), 66 bot-пользователей и 18 оплат. Повторные, organic и unmapped события проверяют дедупликацию. Исходные исторические суммы и даты сохранены; вымышленные и live данные разделены.

Календарь и стабильные ссылки

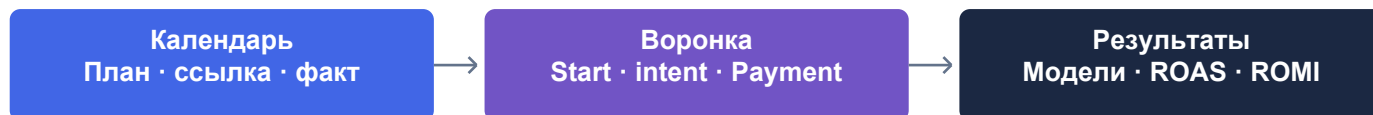


Команда выбирает кампанию, канал, курсы, дату UTC, бюджет и текст. У размещения свой стабильный botTrackingKey. После выпуска нельзя менять кампанию, канал или набор курсов; повторный выпуск сохраняет адрес. Номер будущего поста для ключа не нужен.

Источник telegram|ASCII_channel|short_key кодируется UTF-8 base64url, start не длиннее 64 символов. Сайт сохраняет web_visit перед 302 в @tracker_marketing_bot. Ссылка действует до обнаружения публикации. Preview передаёт test_* и не пишет web-событие; исторические и отменённые ссылки отклоняются.

Автопоиск публичного поста проверяет точную ссылку в окне -15 минут / +2 часа к плану. Сохраняет план и факт отдельно. Проверки: интерфейс каждые 30 секунд, Cron каждые 5 минут. Ручной адрес второстепенный и имеет отдельное основание. Cron ищет публикации, **не загружает CSV бота**.

Три связанных экрана



Публичный доступ и хранение

Cloudflare Workers / D1: просмотр без ChatGPT, изменения по session команды. Бот работает в Cloudflare webhook: ноутбук можно выключить. Управление через кнопку «Бот» после входа в команду. CSV остаётся резервным импортом. Webhook доставляет события прямо в D1. Добавлены служебные таблицы bot_runtime и telegram_updates; предметная модель сохранена.

demo-komandy-postupili.postupili-demo.workers.dev

Домены, платные планы, сервисы и списания не покупались.

Воронка, атрибуция и экономика

События бота и подтверждение оплаты

events.csv: timestamp,user_id,username,event,source_code,social_network,channel,content_id. Резервный импорт: preview → commit, до 2 МБ / 5000 строк; для даты без timezone нужен явный offset. Telegram ID преобразуется HMAC-SHA256 с серверным salt; raw ID, username и CSV не сохраняются. Полный source_code сопоставляется с размещением; неизвестный источник не заимствуется из прошлых событий.

start - наблюдаемый запуск. **payment_click** - намерение купить / передача менеджеру, не оплата. Первый click по (mode,userKey,placementId,sourceCode) создаёт устойчивую PurchaseIntent. Raw события и уникальные люди показаны отдельно; общий итог объединяет людей между размещениями. Browser ID и Telegram userKey индивидуально не склеиваются.

Менеджер создаёт **Payment** по существующей заявке: уникальный ID, курс, сумма, UTC-дата. Один курс подставляется; несколько требуют подтверждённого выбора. Вне бота - явный Unknown и null userKey. Refund уменьшает Net, но сохраняет факт прежней покупки; не меняет starts / intents. Нет оплат - нет финансовых конверсий.

Все четыре модели сохранены

Live/demo кандидаты: **bot start того же userKey**, точное размещение, совместимый курс, до оплаты и внутри окна. Web visit и payment_click не атрибуционные касания. Дедупликация по **placementId**: earliest для First, latest для остальных. Два поста одной кампании остаются двумя размещениями.

First Touch - 100% первому; **Last Touch** - 100% последнему; **Linear** - поровну уникальным размещениям; **Time Decay** - вес $2^{-(\text{дни/полураспад})}$, затем нормировка. Сначала Allocation, после него агрегация и фильтры по кампании / каналу / курсу. Скрытые доли не перенормируются; без подходящего касания деньги остаются Unknown. Это правило распределения, не причинный эффект.

История - только сценарная оценка. Неизвестно, видел ли покупатель конкретный пост. Модели распределяют оплату между совместимыми публикациями, НЕ устанавливают фактический источник. Family-гипотезы опциональны: relevance 0,5 в Linear/Decay; First/Last могут дать 100% с low confidence. Покрытие и задержка описывают гипотезы.

Расчёт денег

Net = сумма - возврат. Доход до маркетинга $C = \text{Net} \times (1 - \text{обучение}\% - \text{комиссия}\%)$ - стоимость обслуживания оплаты. Реклама A суммируется по размещениям; прочий маркетинг O - один раз на кампанию; $M = A + O$. Многокурсовый бюджет делится по долям, иначе поровну с явным допущением.

ROAS = Net / A. ROMI = (C - M) / M × 100%. CPL = M / уникальные заявки; CPA = M / доли подтверждённых платежей, включая возвраты, не CAC. CR start → intent и intent → оплаченный пользователь когорты считаются из уникальных ключей. Неизвестные расходы, нулевой знаменатель и незрелая когорта дают прочерк.

Пример: Net 10 000, обучение 30%, комиссия 2%, обслуживание 500 → $C = 6\,300$. A = 5 000, O = 1 000 → **ROAS 2×, ROMI 5%**. При полном возврате обслуживание остаётся. Постоянные расходы и налоги не распределяются.

Проверки: 67 тестов движка/контракта/воронки/webhook/публикаций/экспорта, 45 локальных API-проверок; typecheck, application lint, production build, внешний smoke. Offline: 8 проверок бота без Telegram-запросов. Реальный Start после настройки токена проверяет команда. События webhook поступают автоматически; CSV сохраняет совместимость.

[Репозиторий команды](#) · [Telegram contract](#) · [Cloudflare](#)